

Solution Requirements

Date	5-7-2026
Team ID	
Project Name	EduGenie Google Gemini Powered Learning Assistant
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	User Access	Users can use features such as Question Answering, Concept Explanation, Quiz, Summary, and Learning Path from a single interface.	High
2	AI Integration	The system uses Google Gemini 2.5 Flash API to generate educational content and responses.	High
3	User Authentication	Users can securely access the application to use the available learning services.	High
4	Data Storage	The application stores user activities, quiz data, summaries, and learning records using SQLite/JSON for future reference.	High
5	Request Processing	FastAPI processes user requests and returns AI-generated results based on the selected feature.	High
6	Result Display	The application presents explanations, answers, quizzes, summaries, and learning paths through an interactive web interface.	Medium
7	Security & Privacy	API credentials and application data are protected using secure configuration and environment variables.	Medium
8	Business Rules	The system validates user input, limits invalid requests, and provides appropriate error messages when needed.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance	AI responses should be generated with minimal delay.	Average response time of 2–5 seconds
2	Usability	The interface should be simple, responsive, and easy for students to navigate.	Mobile-friendly and intuitive UI
3	Scalability	The application should support additional users and future feature enhancements.	Supports 100+ concurrent users
4	Reliability & Availability	The system should remain stable during normal usage with minimal downtime.	99% availability
5	Security	Sensitive information such as API keys must be securely stored and protected.	Secure .env configuration and protected credentials
6	Maintainability	The project should follow a modular structure so new features can be added easily.	Clean, reusable, and well-organized code